

GITTO

Goal Tracking App

AXEL VERDUGO,
JENNIFER IMAFIDON,
BIBEK K C

WHAT IS GITTO & PURPOSE?

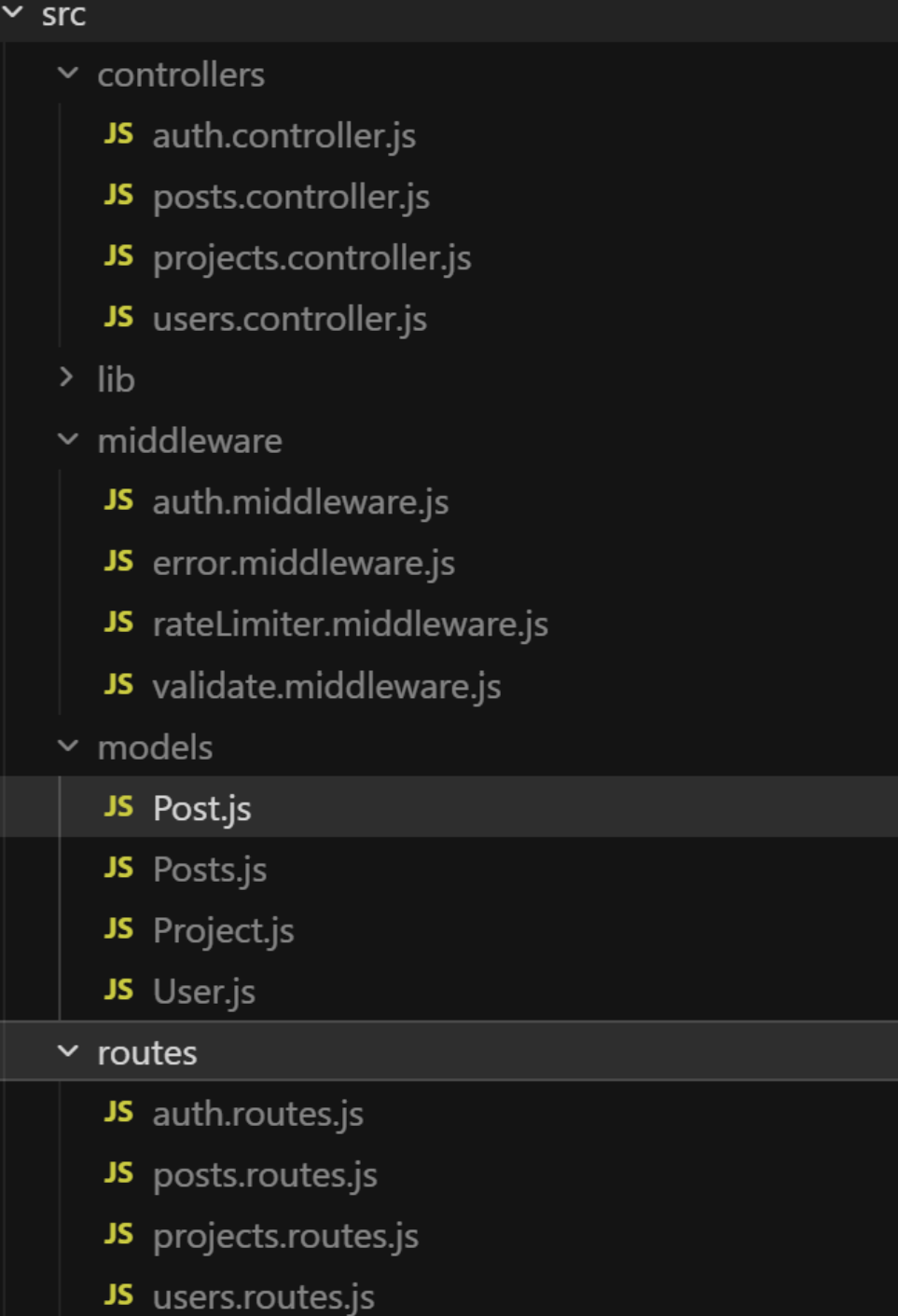
- Gitto is a web-application created to track progress in a wide range of projects, while also being a social media platform.
 - Allowing users to create projects without real topic/specialization restrictions and keep people updated on its progress. Since there are no restrictions on project type or theme, any person with any interests may create projects.
 - Allows people to follow closely on the progress of projects they are interested in.
 - Users can create posts to celebrate a milestone, or just to update people on the general progress of the project itself.
 - Users can leave critiques or support under each project.
-

MAIN COMPONENTS

- Our application was composed of **5 major components**, including the **Login Page**, **Feed Page**, **Project Page**, **Search Page**, and **User Page**. Each of these pages had to be available for the user to have the complete Gitto experience.
 - The flow of the application starts with the Login Page, which has OAuth available as an option for users. This takes you to:
 - The **Feed Page and Project Page** had most of the digestible content that the site offered. The Feed Page highlights the milestone posts of followed users, as well as other progress posts. Interacting with these posts could take the user to the post's Project Page or author's User Page.
-

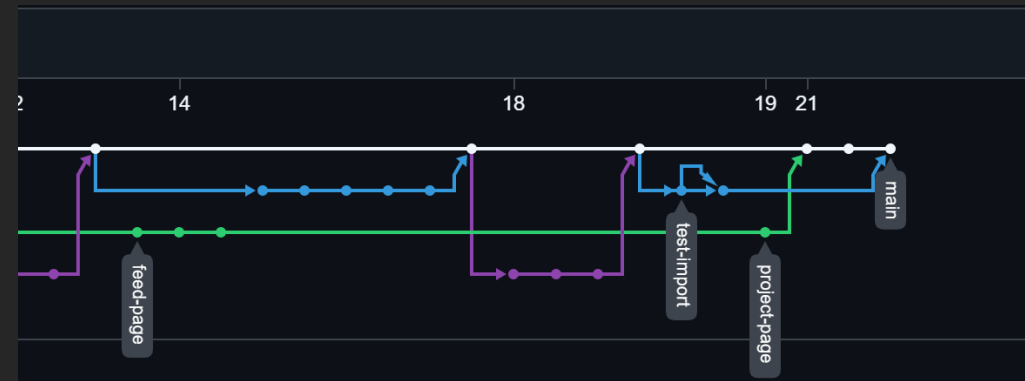
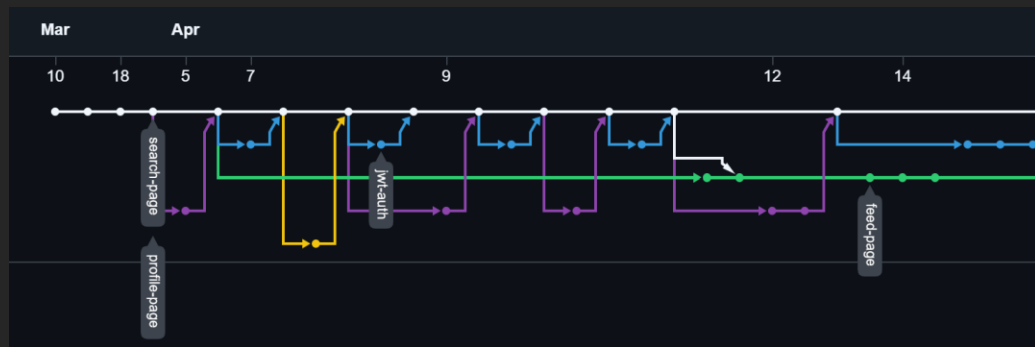
DATABASE STRUCTURE

- Since Gitto is a social media platform, it requires managing a great influx of data, **post data, user data, project data**.
 - For our project we opted for using a mongoose database. We created three major schema.
 - User Schema, which has all user information, including but not limited to: Name, handle, email, **nullable** password, bio if any, users followed, followers and more. The password is nullable since OAuth is an option for users.
 - **Project Schema**, which was divided into three different schemas, **Project Schema, Project Comment Schema, and Project Milestone schema**.
 - Finally **Post Schema**, which was also divided into **Post Schema, Post Comment Schema and Post Attachment Schema**.
-



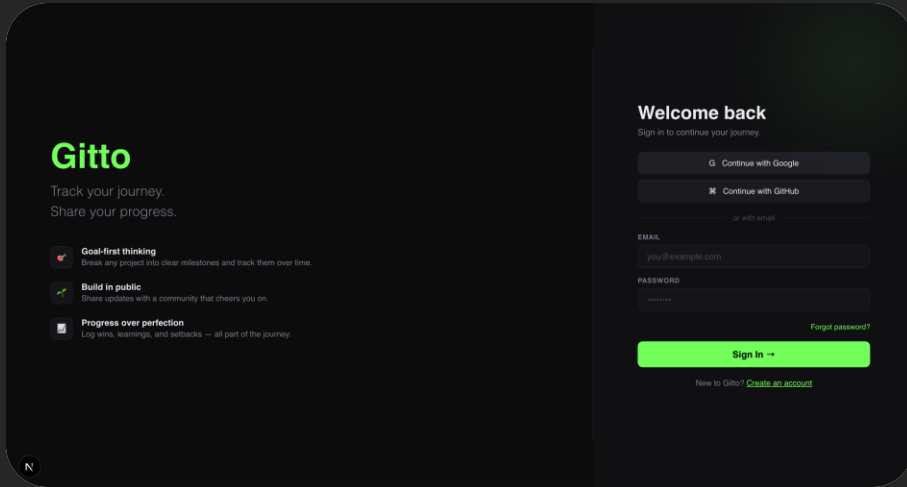
MVC STRUCTURE

- Our project closely adhered to MVC structure, separating our backend and frontend into Controller, Visual, and Model sections. Ensuring the flow of information and site navigation was as consistent as possible.
- Our model layer holds all the schema and data rules
- Controller layer handles requests and feature logic

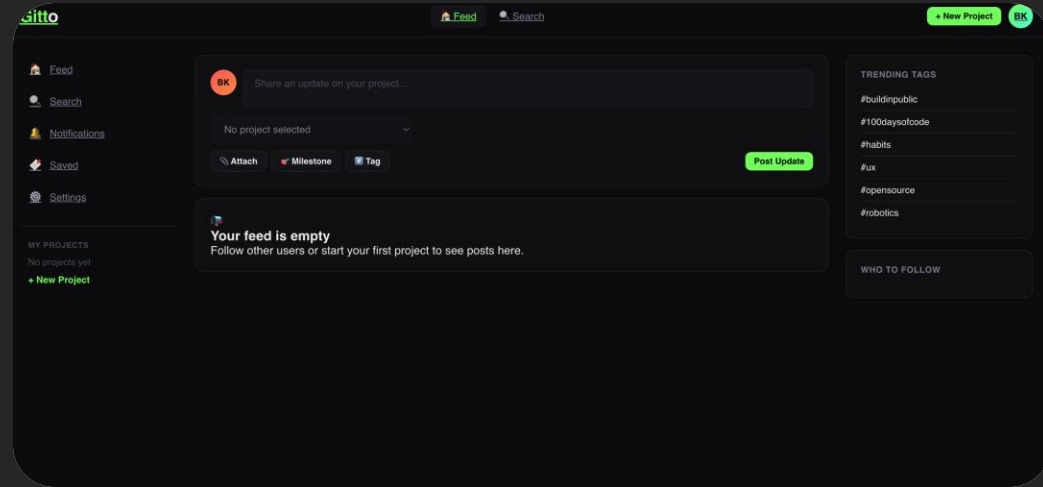


COMNETWORK

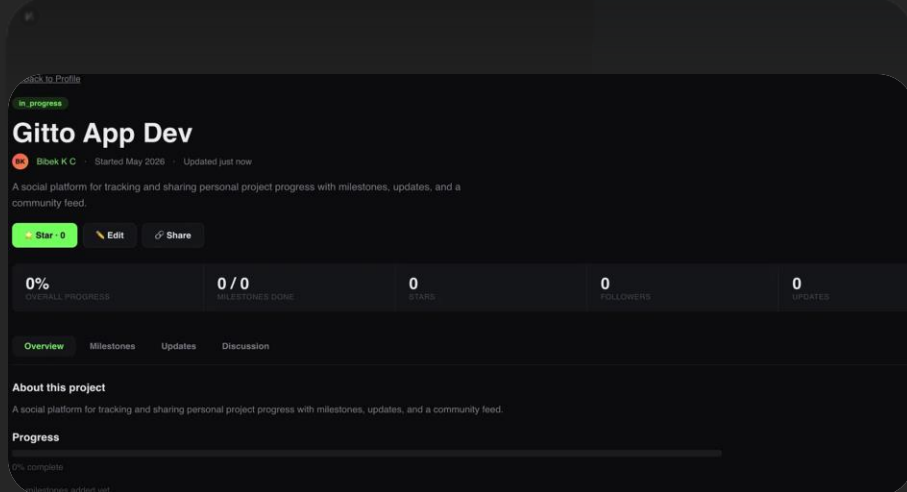
TEST CASES



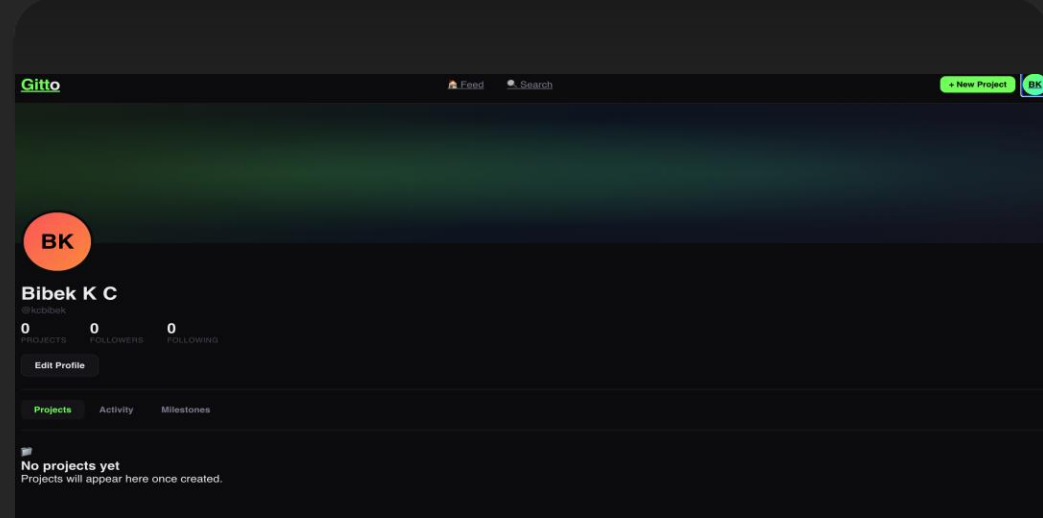
• Login Page



• Feed Page



• Project Page



• User Page

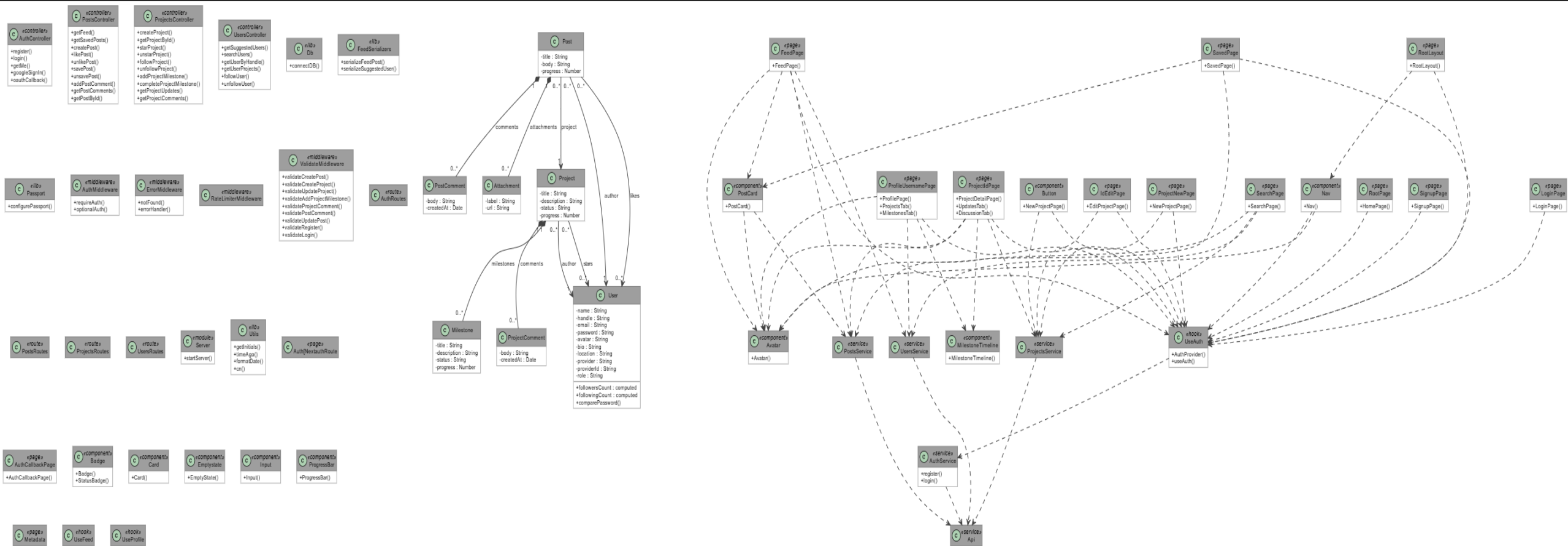
Test Result

```
✓ api/feature-routes.test.js (4) 5584ms
✓ api/health.test.js (1) 5210ms
✓ api/jwt.auth.test.js (9) 31034ms
✓ api/users.test.js (27) 79225ms

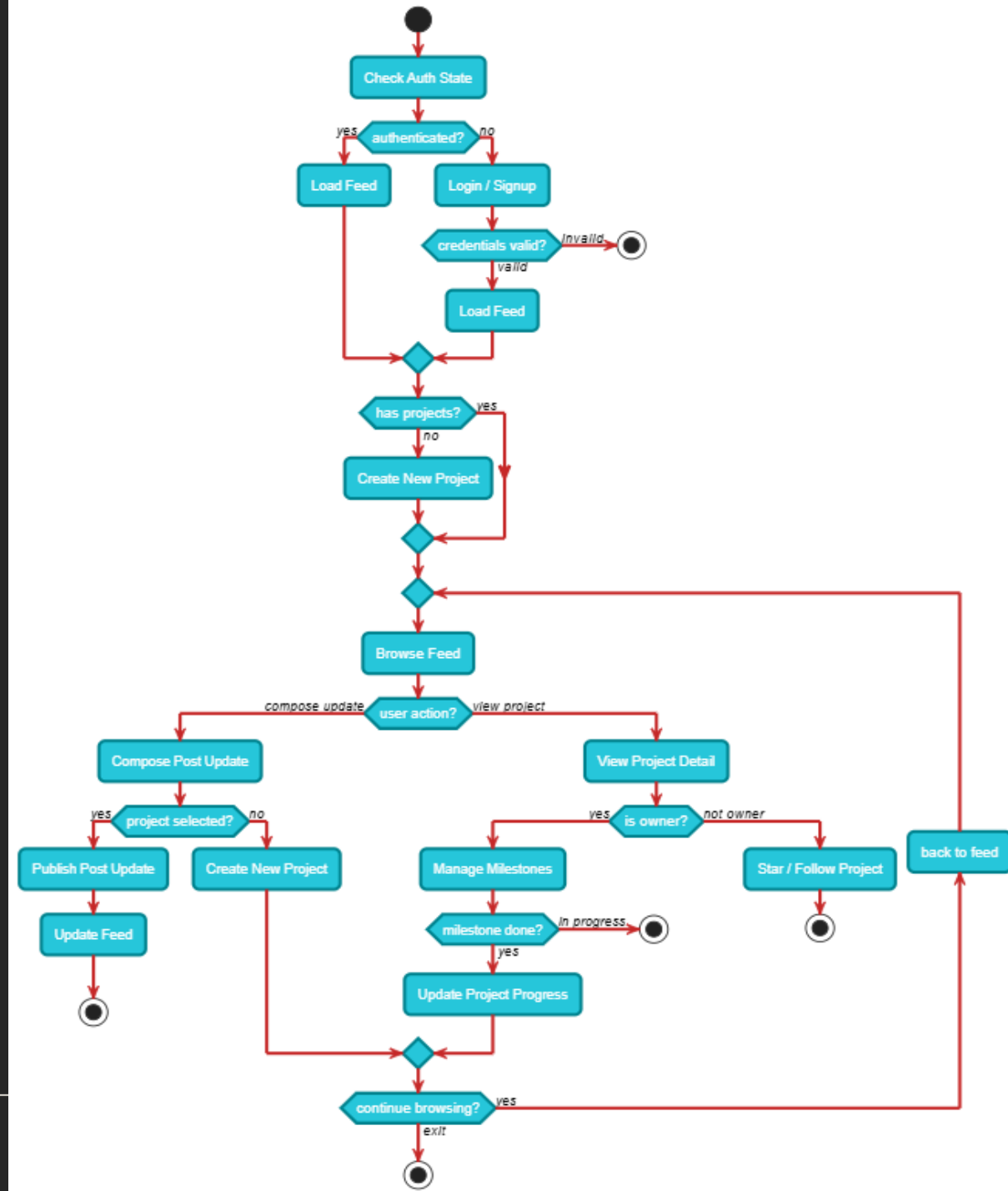
Test Files  4 passed (4)
   Tests   41 passed (41)
   Start at 07:47:09
   Duration 162.78s (transform 1.00s, setup 13.49s, collect 16.54s, tests 121.05s, environment 6ms, prepare 3.03s)

Dockerfile 1e5.18s (transform 1.00s, setup 13.49s, collect 16.54s, tests 121.05s, environment 6ms, prepare 3.03s)
21911 91 03:43:00
```

- Unit tests were run using Vitest and Supertest across 4 test files. All 41 tests passed with 0 failures, covering MongoDB connection, API route validation, JWT authentication, user registration, login flows, and protected route access control.



System Statechart Diagram



Thanks
